

JAVA WEB APPLICATION TECHNOLOGIES

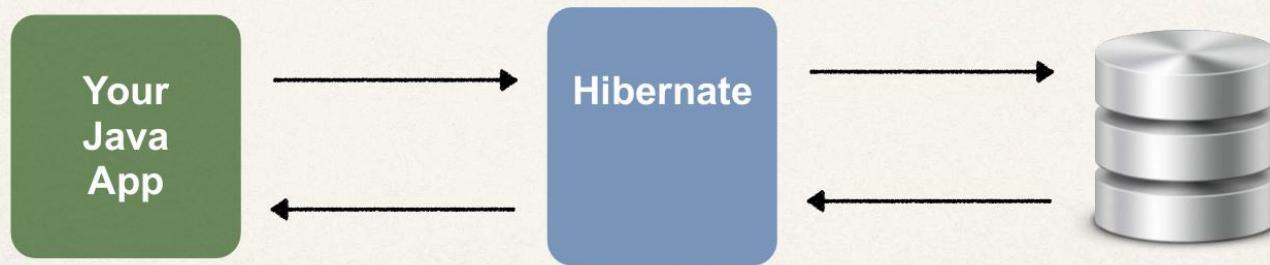


Hibernate



What is Hibernate

- A framework for persisting / saving Java objects in a database
- www.hibernate.org/orb



Hibernate

Hibernate is a Java framework that simplifies the development of Java application to interact with the database. It is an open source, lightweight, ORM (Object Relational Mapping) tool. HAn object-relational mapping (ORM) library for Java

- Allows us to interact with a database using regular Java objects
- Provides a closer relationship between your Java objects and tables in your database
- Allows us to easily persist objects in a database
- Works with different database types, and so is an abstraction layer
- Hibernate implements the specifications of JPA (Java Persistence API) for data persistence



ORM

- ORM stands for **O**bject-**R**elational **M**apping (ORM) is a programming technique for converting data between relational databases and object oriented programming languages such as Java, C#, etc.



What is JPA?

- The Java Persistence API (JPA) is a specification of Java. It is used to persist data between Java object and relational database. JPA acts as a bridge between object-oriented domain models and relational database systems.
- As JPA is just a specification, it doesn't perform any operation by itself. It requires an implementation. So, ORM tools like Hibernate, TopLink and iBatis implements JPA specifications for data persistence.



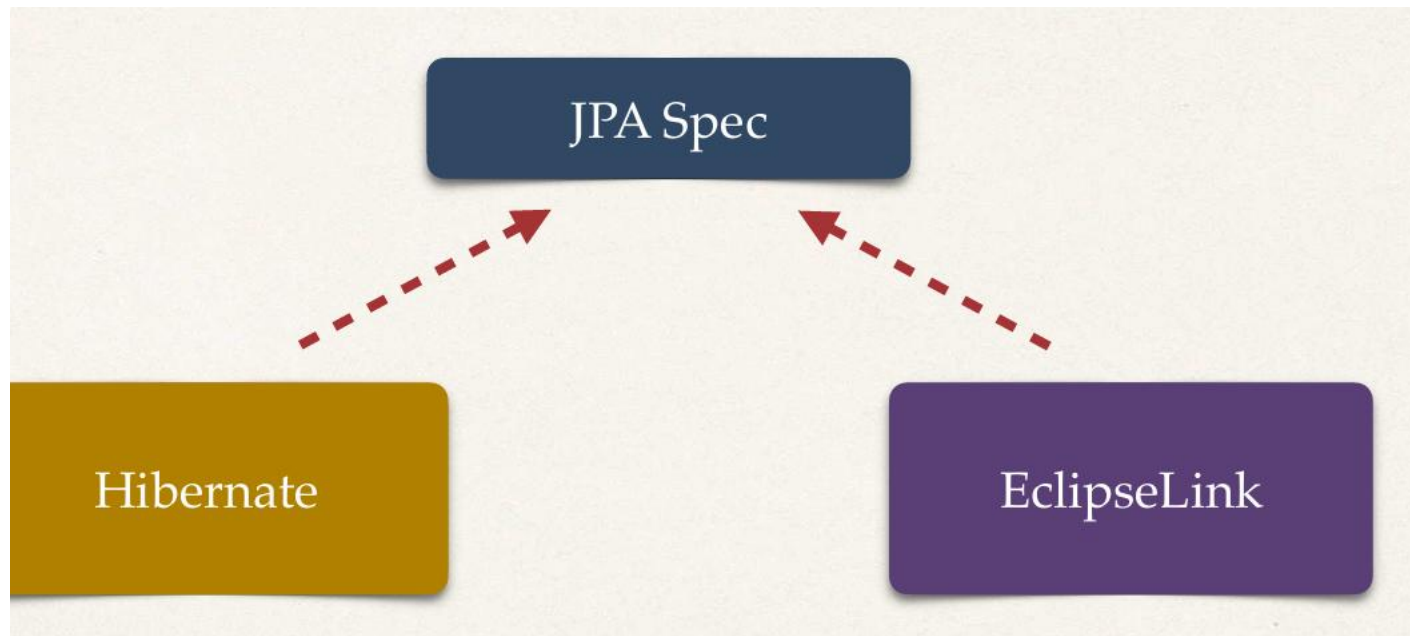
What are Benefits of JPA

- By having a standard API, you are not locked to vendor's implementation
- Maintain portable, flexible code by coding to JPA spec (interfaces)
- Can theoretically switch vendor implementations
 - For example, if Vendor ABC stops supporting their product
 - You could switch to Vendor XYZ without vendor lock in



JPA - Vendor Implementations

- For more, read: https://en.wikipedia.org/wiki/Jakarta_Persistence



Advantages of Hibernate

- 1) Open Source and Lightweight:** Hibernate framework is open source under the LGPL license and lightweight.
- 2) Fast Performance:** The performance of hibernate framework is fast because cache is internally used in hibernate framework. There are two types of cache in hibernate framework first level cache and second level cache. First level cache is enabled by default.
- 3) Database Independent Query:** HQL (Hibernate Query Language) is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, if database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem.
- 4) Automatic Table Creation:** Hibernate framework provides the facility to create the tables of the database automatically. So there is no need to create tables in the database manually.
- 5) Simplifies Complex Join:** Fetching data from multiple tables is easy in hibernate framework.
- 6) Provides Query Statistics and Database Status:** Hibernate supports Query cache and provide statistics about query and database status.



Hibernate Architecture

❖ The Hibernate architecture includes many objects such as :

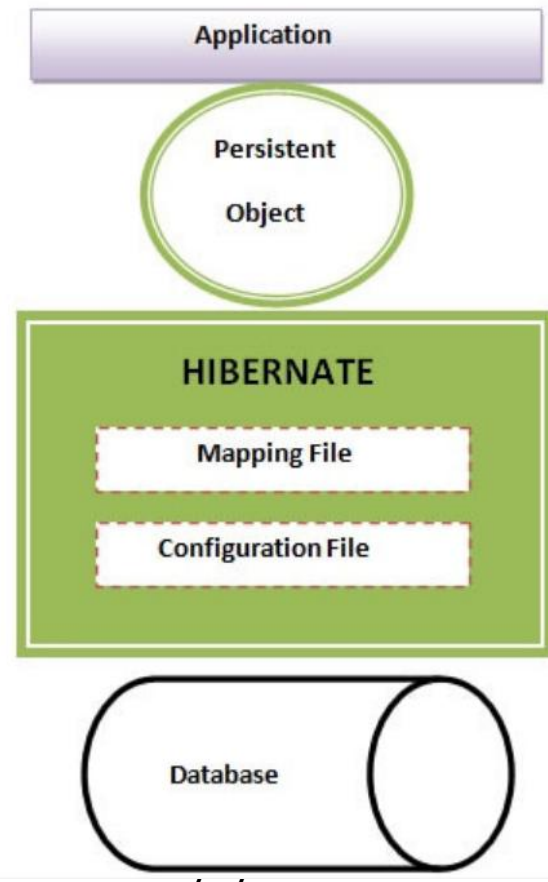
- persistent object,
- session factory,
- transaction factory,
- connection factory,
- Session,
- transaction etc.



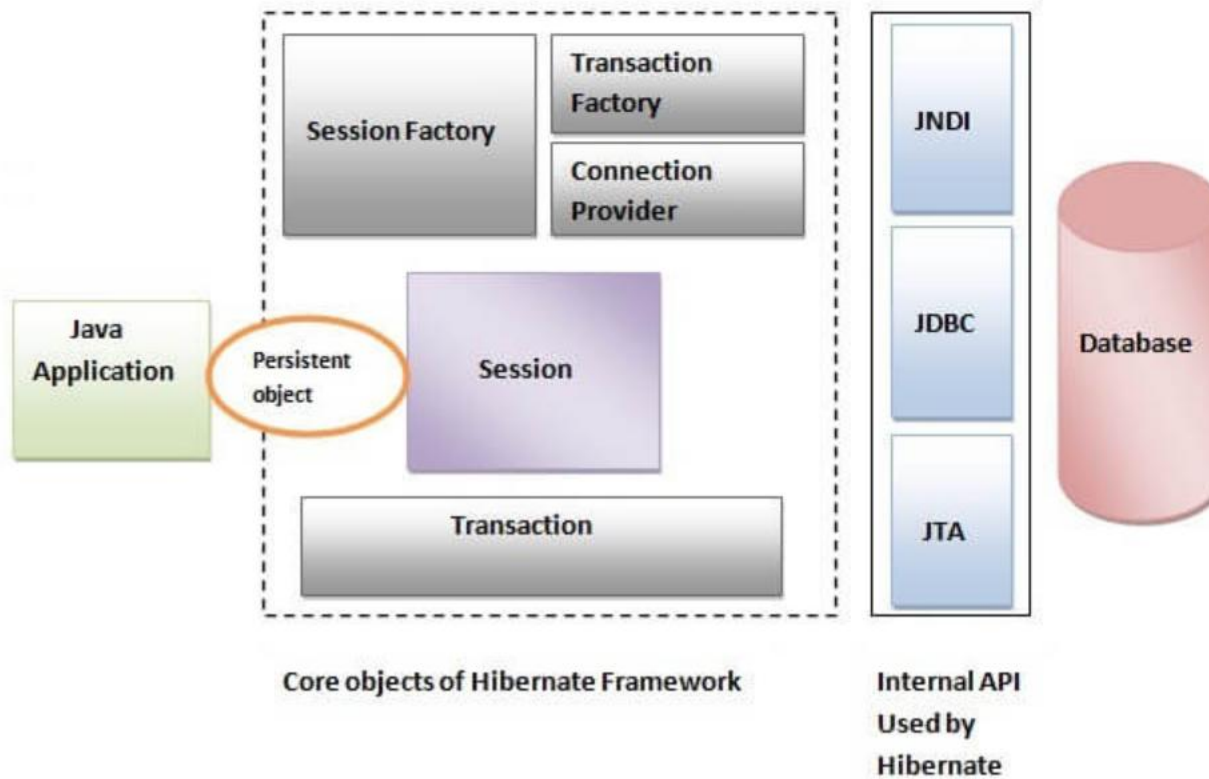
Layers of Hibernate Architecture

- The Hibernate architecture is categorized in four layers.

- Java application layer
- Hibernate framework layer
- Backhand api layer
- Database layer



Hibernate Architecture



Element of Hibernate Architecture

- **SessionFactory:**

- ❖ The SessionFactory is a factory of session and client of ConnectionProvider

- **Session**

- ❖ The session object provides an interface between the application and data stored in the database
- ❖ It is factory of Transaction, Query and Criteria.
- ❖ The org.hibernate.Session interface provides methods to insert, update and delete the object. It also provides factory methods for Transaction, Query and Criteria.



Element of Hibernate Architecture

- **Transaction:**

- ❖ The `org.hibernate.Transaction` interface provides methods for transaction management.

- **ConnectionProvider**

- ❖ It is a factory of JDBC connections. It abstracts the application from `DriverManager` or `DataSource`. It is optional.



Configuration

- Hibernate requires to know in advance — where to find the mapping information that defines how your Java classes relate to the database tables.
- Hibernate also requires a set of configuration settings related to database and other related parameters. All such information is usually supplied as a standard Java properties file called **hibernate.properties**, or as an XML file named **hibernate.cfg.xml**.



Hibernate properties

Sr.No.	Properties & Description
1	hibernate.dialect This property makes Hibernate generate the appropriate SQL for the chosen database.
2	hibernate.connection.driver_class The JDBC driver class.
3	hibernate.connection.url The JDBC URL to the database instance.
4	hibernate.connection.username The database username.
5	hibernate.connection.password The database password.
6	hibernate.connection.pool_size Limits the number of connections waiting in the Hibernate database connection
7	hibernate.connection.autocommit Allows autocommit mode to be used for the JDBC connection.



Example of configuration file using XML

```
-//Hibernate/Hibernate Configuration DTD 3.0//EN (doctype with catalog)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/employeeDetails</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.connection.autocommit">>false</property>
    <property name="hibernate.show_sql">>true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <mapping resource="Employee.hbm.xml"/>
    <mapping class="com.models.Student"/>
  </session-factory>
</hibernate-configuration>
```



Object Mapping

- Tell Hibernate how to map a Java class to a database table
- Hibernate can then
 - Save instances as new table rows
 - Load instances of that class from rows in the table
 - Update instances
- We can use XML files or annotations



Mapping using XML

- XML-based configuration is one way to define mappings between Java classes and database tables. The mapping is usually defined in a file with a **.hbm.xml** extension.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE hibernate-mapping PUBLIC
3      "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4      "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
5  <hibernate-mapping package="com.Models" >
6  <class name="Student" table="students">
7      <!-- Primary key mapping -->
8      <id name="studentId" column="studId"/>
9
10
11      <property name="firstName" column="first_name"/>
12      <property name="lastName" column="last_name"/>
13
14  </class>
15  </hibernate-mapping>
16
```



Mapping using annotations

Annotations allow you to map entities and their relationships without xml files.

- All the mapping information will be in the code for each domain/model object
- Hibernate annotations are the newest way to define mappings without the use of XML file. You can use annotations in addition to or as a replacement of XML mapping metadata.
- Hibernate Annotations is the powerful way to provide the metadata for the Object and Relational Table mapping. All the metadata is clubbed into the POJO java file along with the code, this helps the user to understand the table structure and POJO simultaneously during the development.
- Hibernate annotations for POJO are contained in JPA through **jakarta.persistence** package. The old version of the package was `javax.persistence`.

Src: <https://docs.jboss.org/hibernate/jpa/2.2/api/javax/persistence/package-summary.html>



Creating domain objects: classes

```
public class Student {  
    private Long id;  
    private String firstName;  
    private String lastName;  
    private String email;  
    private int age;  
  
    public Student() {  
    }  
    public Student(long id, String firstName, String lastName,  
String email, int age) {  
        this.id = id;  
        this.firstName = firstName;  
        this.lastName = lastName;  
        this.email = email;  
        this.age = age;  
    }  
    public Student(String firstName, String lastName, String email,  
int age) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
        this.email = email;  
        this.age = age;  
    }  
    public void setId(long id) {  
        this.id = id;  
    }  
}
```

Hibernate requires
classes with default
constructors

We can also define an
explicit constructor
for our use

All properties are
accessed through
getter and setter
methods



What is a POJO?

POJO stands for **Plain Old Java Object**. POJO stands for Plain Old Java Object. In simple terms, we use POJO to make a programming model for declaring object entities.

Properties of POJO classes

1. The POJO classes must be public so that we can use them outside the class.
2. POJO does not have any name convention for properties and methods. It's just a java class consisting of some variables and getters-setters.
3. POJO classes are used in hibernate for mapping to database objects. That means all object entities we make in POJO classes will be reflected in a database object.
4. It should not extend classes, implement interfaces, or contain prespecified annotations. Also, it does not bound with any restrictions but strictly follows java language specifications.
5. In hibernate, POJOs also contain some annotations like `@Entity`, `@Table`, `@id`, etc for avoiding XML files for database objects.



Hibernate Annotations

- **@Entity**

Annotate all your entity beans with @Entity. This means the annotated class is a persistent entity and hibernate will map the class to a table in the database.

@Entity

```
public class Company implements Serializable {  
}
```

@Entity

```
public class Company {  
}
```

- **@Table**

Specify the database table this Entity maps to using the name attribute of @Table annotation. In the example below, the data will be stored in 'companies' table in the database.



Hibernate Annotations

@Entity

@Table(name = "companies")

```
public class Company implements Serializable {  
}
```

The `@Table` annotation allows you to specify the details of the table that will be used to persist the entity in the database.

The `@Table` annotation provides four attributes, allowing you to override the name of the table, its catalogue, and its schema, and enforce unique constraints on columns in the table



More about @Table annotation

- To rename the key like Unique Key Constraint, use @Table annotation

```
@Table(name = "student",  
uniqueConstraints =  
{@UniqueConstraint(name="st_email",columnNames="email")})  
Class Student{}
```

If the style is used, the unique=true of the @column annotation for email is removed.

Generating a constraint for two or more columns

```
@Table( name = "MYTABLE", uniqueConstraints = { @UniqueConstraint(  
columnNames = { "STUDENTID", "COURSEID", "TERMID" } ) } )
```



Hibernate annotations

- ***Id* Annotation**

Each JPA entity must have a primary key that uniquely identifies it. The `@Id` annotation defines the primary key. We can generate the identifiers in different ways, which are specified by the `@GeneratedValue` annotation.

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;

    private String name;

    // getters and setters
}
```



Hibernate annotations

- Id Generation strategy

Enum Constant	Description
<u>AUTO</u>	Indicates that the persistence provider should pick an appropriate strategy for the particular database. This is the default if not specified strategy.
<u>IDENTITY</u>	This strategy relies on the auto-increment functionality provided by the database to generate unique identifier values automatically. Identity columns are supported in many databases, such as MySQL, DB2, SQL Server, Sybase and Postgres. Oracle does not support IDENTITY columns but they can be simulated through using sequence objects and triggers.
<u>SEQUENCE</u>	With this strategy, the <code>@GeneratedValue</code> annotation fetches unique identifier values from a predefined sequence generator. Usually, a SEQUENCE object has a name, an INCREMENT, and other database object settings. Each time the <code>.NEXTVAL</code> is selected the sequence is incremented by the INCREMENT.
<u>TABLE</u>	Indicates that the persistence provider must assign primary keys for the entity using an underlying database table to ensure uniqueness.
UUID	Indicates that the persistence provider must assign primary keys for the entity by generating an RFC 4122 Universally Unique <u>Identifier</u> .



Hibernate annotations

- **The *Column* Annotation**

- Just like the *@Table* annotation, we can use the *@Column* annotation to mention the details of a column in the table.
- The *@Column* annotation has many elements such as *name*, *length*, *nullable*, and *unique*:

```
@Entity
@Table(name="STUDENT")
public class Student {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;

    @Column(name="STUDENT_NAME", length=50, nullable=false, unique=false)
    private String name;

    // other fields, getters and setters
}
```



Hibernate annotations

- **The *Transient* Annotation**

- Sometimes, we may want to **make a field non-persistent**. We can use the `@Transient` annotation to do so. It specifies that the field won't be persisted.
- For instance, we can calculate the age of a student from the date of birth.

```
@Entity
@Table(name="STUDENT")
public class Student {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;

    @Column(name="STUDENT_NAME", length=50, nullable=false)
    private String name;

    @Transient
    private Integer age;

    // other fields, getters and setters
}
```

Adva



Hibernate annotations

- **The *Temporal* Annotation**

- The **@Temporal** is a Hibernate JPA annotation that is used for **java.util.Date** or **java.util.Calendar**. It converts the date and time values into a compatible format when storing or retrieving from a database. When handling temporal data remember to define the expected precision in the database. Temporal data precision can have **DATE**, **TIME**, or **TIMESTAMP**.

```
@Temporal(TemporalType.TIMESTAMP)
@Column(name="DATETIME_COL")
public Date dateAndTime;
```

```
@Temporal(TemporalType.TIMESTAMP)
@Column(name="TIMESTAMP_COL")
public Date timeStamp;
```

```
@Temporal(TemporalType.DATE)
@Column(name="DATE_COL")
public Date date;
```

```
@Temporal(TemporalType.TIME)
@Column(name="TIME_COL")
```



Mapping Java Classes to Database Tables:

1. Inheritance Mapping

- Hibernate supports three main strategies for mapping an inheritance hierarchy:

1. Table per Class Hierarchy (Single Table Inheritance)

2. Table per Subclass (Joined Table Inheritance)

3. Table per Concrete Class (Table per Class)



1. Table per Class Hierarchy (Single Table Inheritance)

- In this strategy, a single table is used to store all the instances of the inheritance hierarchy. Each record contains a discriminator column to identify the subclass it belongs to.
- **Advantages**
 - Simplicity: Only one table to manage.
 - Performance: Fewer joins required for queries.
- **Disadvantages**
 - Null values: Columns for attributes not used by some subclasses will be null.
 - Database schema may not be normalized.



Example of using a single table

```
import jakarta.persistence.*;
```

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "vehicle_type",
discriminatorType = DiscriminatorType.STRING)
public class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String manufacturer;
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getManufacturer() {
        return manufacturer;
    }
    public void setManufacturer(String manufacturer) {
        this.manufacturer = manufacturer;
    }
}
```

```
package com.models;
import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;

@Entity
@DiscriminatorValue("Car")

public class Car extends Vehicle {
    private int numDoors;

    public int getNumDoors() {
        return numDoors;
    }

    public void setNumDoors(int numDoors) {
        this.numDoors = numDoors;
    }
}
```



Example of using a single table

```
package com.models;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("Bike")
public class Bike extends Vehicle {

    private boolean hasPedals;

    public boolean isHasPedals() {
        return hasPedals;
    }

    public void setHasPedals(boolean hasPedals) {
        this.hasPedals = hasPedals;
    }
}
```

				vehicle_type	id	manufacturer	numDoors	hasPedals
☐	 Edit	 Copy	 Delete	Car	1	Toyota	4	NULL
☐	 Edit	 Copy	 Delete	Bike	2	Samsung	NULL	1



Table per Subclass (Joined Table Inheritance)

- Each class in the hierarchy is mapped to its own table. The base class table contains common attributes, while subclass tables have specific attributes.
- **Advantages**
 - No null values: Each table has only relevant columns.
 - Better normalization: Each table represents a distinct entity.
- **Disadvantages**
 - Performance: Joins are required to retrieve subclass objects.
 - Complexity: More tables to manage.



Joined Table Inheritance (Example)

```
import jakarta.persistence.*;

@Entity
@Inheritance(strategy = InheritanceType.JOINED)
@DiscriminatorColumn(name = "vehicle_type",
discriminatorType = DiscriminatorType.STRING)
public class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String manufacturer;
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getManufacturer() {
        return manufacturer;
    }
    public void setManufacturer(String manufacturer) {
        this.manufacturer = manufacturer;
    }
}
```

```
package com.models;
import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;

@Entity

public class Car extends Vehicle {
    private int numDoors;

    public int getNumDoors() {
        return numDoors;
    }

    public void setNumDoors(int numDoors) {
        this.numDoors = numDoors;
    }
}
```



Joined Table Inheritance (Example)

```
1 package com.models;
2
3 import jakarta.persistence.*;
4
5
6 @Entity
7
8 public class Bike extends Vehicle {
9
10     private boolean hasPedals;
11
12     public boolean isHasPedals() {
13         return hasPedals;
14     }
15
16     public void setHasPedals(boolean hasPedals) {
17         this.hasPedals = hasPedals;
18     }
19
20 }
```



Joined Table Inheritance (Example)

←T→				vehicle_type	id	manufacturer
☐	 Edit	 Copy	 Delete	Car	1	Toyota
☐	 Edit	 Copy	 Delete	Bike	2	Samsung

Vehicle Table

				hasPedals	id
☐	 Edit	 Copy	 Delete	1	2

Bike Table

				numDoors	id
☐	 Edit	 Copy	 Delete	4	1

Car Table



Table per Concrete Class (Table per Class)

- Each class, including the abstract superclass, is mapped to its own table. There is no table for the abstract superclass, and each table includes columns for all inherited properties.
- **Advantages**
 - Simple to implement: Each class directly maps to a table.
- **Disadvantages**
 - Redundancy: Common attributes are repeated in multiple tables.
 - Query complexity: Queries involving the superclass require unions.



Table per Concrete Class (Example)

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)

public class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    private Long id;
    private String manufacturer;
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getManufacturer() {
        return manufacturer;
    }
    public void setManufacturer(String manufacturer) {
        this.manufacturer = manufacturer;
    }
}
```



Association Mapping in Hibernate

- Association mapping in Hibernate is a crucial concept that helps in defining relationships between entities in a relational database. Understanding how to map these associations is essential for effective database management and data retrieval in Hibernate ORM (Object-Relational Mapping).



Unidirectional Associations

- Unidirectional associations are commonly used in object-oriented programming to establish relationships between entities. **However, it's important to note that in a unidirectional association, only one entity holds a reference to the other.**
- To define a unidirectional association in Java, we can use annotations such as *@ManyToOne*, *@OneToMany*, *@OneToOne*, and *@ManyToMany*. By using these annotations, we can create a clear and well-defined relationship between two entities in our code.



Unidirectional Associations:1. One-To-Many Relationship

- In a one-to-many relationship, an entity has a reference to one or many instances of another entity.
- A common example is the relationship between a *Department* and its *Employees*. Each *Department* has many *Employees*, but each *Employee* belongs to one *Department* only.



Unidirectional Associations: 1. One-To-Many Relationship

```
@Entity
public class Department {

    @Id
    private Long id;

    @OneToMany
    @JoinColumn(name = "department_id")
    private List<Employee> employees;
}

@Entity
public class Employee {

    @Id
    private Long id;
```

- Here, the *Department* entity has a reference to a list of *Employee* entities. The `@OneToMany` annotation specifies that this is a one-to-many association. The `@JoinColumn` annotation specifies the foreign key column in the *Employee* table referencing the *Department* table.



Unidirectional Associations:2. Many-To-One Relationship

- **In a many-to-one relationship, many instances of an entity are associated with one instance of another entity.**
- For example, let's consider *Student* and *School*. Each *Student* can be enrolled in one *School* only, but each *School* can have multiple *Students*.

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne
    @JoinColumn(name = "school_id")
    private School school;
}
```

```
@Entity
public class School {
```



```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

Unidirectional Associations:3. One-To-One Relationship

- In a **one-to-one** relationship, an instance of an entity is associated with only one instance of another entity..
- A common example is the relationship between an *Employee* and a *ParkingSpot*. Each *Employee* has a *ParkingSpot* and each *ParkingSpot* belongs to one *Employee*

```
@Entity
public class Employee {

    @Id
    private Long id;

    @OneToOne
    @JoinColumn(name = "parking_spot_id")
    private ParkingSpot parkingSpot;

}

@Entity
public class ParkingSpot {

    @Id
    private Long id;

}
```



Unidirectional Associations:3. Many-To-Many Relationship

- In a **many-to-many** relationship, many instances of an entity are associated with many instances of another entity
- Suppose we have two entities – *Book* and *Author*. Each *Book* can have multiple *Authors*, and each *Author* can write multiple *Books*. In JPA, this relationship is represented using the `@ManyToMany` annotation.



Unidirectional Associations:3. Many-To-Many Relationship

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String title;

@ManyToMany
@JoinTable(name = "book_author",
    joinColumns = @JoinColumn(name = "book_id"),
    inverseJoinColumns = @JoinColumn(name = "author_id"))
private Set<Author> authors;
}

@Entity
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

- Here, we can see a many-to-many unidirectional association between *Book* and *Author* entities. The `@ManyToMany` annotation specifies that each *Book* can have multiple *Authors*, and each *Author* can write multiple *Books*. The `@JoinTable` annotation specifies the name of the join table and the foreign key columns to join the *Book* and *Author* entities



Bidirectional Associations

- **A bidirectional association is a relationship between two entities where each entity has a reference to the other.**
- In order to define bidirectional associations, we use the *mappedBy* attribute in the *@OneToMany* and *@ManyToMany* annotations. However, it's important to note that only relying on unidirectional associations may not be sufficient, as bidirectional associations provide additional benefits.



One-To-Many Bidirectional Association

- In a one-to-many bidirectional association, an entity has a reference to another entity. Additionally, the other entity has a collection of references to the first entity.
- For instance, a *Department* entity has a collection of *Employee* entities. Meanwhile, an *Employee* entity has a reference to the *Department* entity it belongs to.

```
@Entity
public class Department {

    @OneToMany(mappedBy = "department")
    private List<Employee> employees;

}

@Entity
public class Employee {

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

}
```



2/6/2026

One-To-Many Bidirectional Association

- In the *Department* entity, we use the `@OneToMany` annotation to specify the relationship between the *Department* entity and the *Employee* entity. The `mappedBy` attribute specifies the name of the attribute in the *Employee* entity that owns the relationship. In this case, the *Department* entity doesn't own the relationship, so we specify `mappedBy = "department"`.
- In the *Employee* entity, we use the `@ManyToOne` annotation to specify the relationship between the *Employee* entity and the *Department* entity. The `@JoinColumn` annotation specifies the name of the foreign key column in the *Employee* table referencing the *Department* table.



Many-To-Many Bidirectional Association

- **When dealing with a many-to-many bidirectional association, it's important to understand that each entity involved will have a collection of references to the other entity.**
- To illustrate this concept, let's consider the example of a *Student* entity that has a collection of *Course* entities and a *Course* entity that in turn has a collection of *Student* entities. By establishing such a bidirectional association, we enable both entities to be aware of each other and make it easier to navigate and manage their relationship.



Many-To-Many Bidirectional Association

- **When dealing with a many-to-many bidirectional association, it's important to understand that each entity involved will have a collection of references to the other entity.**
- To illustrate this concept, let's consider the example of a *Student* entity that has a collection of *Course* entities and a *Course* entity that in turn has a collection of *Student* entities. By establishing such a bidirectional association, we enable both entities to be aware of each other and make it easier to navigate and manage their relationship.



Many-To-Many Bidirectional Association

```
@Entity
public class Student {

    @ManyToMany(mappedBy = "students")
    private List<Course> courses;

}

@Entity
public class Course {

    @ManyToMany
    @JoinTable(name = "course_student",
        joinColumns = @JoinColumn(name = "course_id"),
        inverseJoinColumns = @JoinColumn(name = "student_id"))
    private List<Student> students;

}
```



Many-To-Many Bidirectional Association

- In the *Student* entity, we use the `@ManyToMany` annotation to specify the relationship between the *Student* entity and the *Course* entity. The `mappedBy` attribute specifies the attribute's name in the *Course* entity that owns the relationship. In this case, the *Course* entity owns the relationship, so we specify `mappedBy = "students"`.
- In the *Course* entity, we use the `@ManyToMany` annotation to specify the relationship between the *Course* entity and the *Student* entity. The `@JoinTable` annotation specifies the name of the join table that stores the relationship.



Embedded Objects in Hibernate

- Embedded objects in Hibernate allow you to define a reusable component class that can be embedded within other entity classes. This is useful when you have a common set of fields that can be reused across multiple entities. Instead of duplicating these fields in every entity class, you define them once in an embeddable class.
- **Key Concepts**
 1. **Embeddable Class:** This is a class that can be embedded in an entity class. It is annotated with **@Embeddable**.
 2. **Embedded Field:** This is a field in the entity class that holds the embeddable object. It is annotated with **@Embedded**.



Embedded Objects in Hibernate(Example)

```
@Embeddable
public class ContactPerson {

    private String firstName;

    private String lastName;

    private String phone;

    // standard getters, setters
}
```

```
@Entity
public class Company {

    @Id
    @GeneratedValue
    private Integer id;

    private String name;

    private String address;

    private String phone;

    @Embedded
    private ContactPerson contactPerson;

    // standard getters, setters
}
```



Benefits of Using Embedded Objects

- 1.Reusability:** Commonly used fields can be encapsulated in a single class and reused across multiple entity classes.
- 2.Maintainability:** Changes to the fields need to be made only in the embeddable class, not in each entity class.
- 3.Clarity:** Embeddable classes can help make your entity classes cleaner and more understandable by grouping related fields together.



Lazy Fetching

- Lazy fetching is a strategy where data is loaded on demand, that is, when it is specifically requested.
- **Key Characteristics:**
- **On-Demand Loading:** Data is fetched only when it is needed. For example, an entity's associated data (e.g., related rows in another table) is not loaded until it is explicitly accessed.
- **Initial Performance Benefit:** Faster initial load times since unnecessary data is not retrieved upfront.
- **Reduced Memory Usage:** Less memory is consumed initially because only the required data is loaded.



Eager Fetching

- Eager fetching is a strategy where related data is loaded at the same time as the initial query.
- **Key Characteristics:**
- **Immediate Loading:** All related data is fetched upfront along with the primary entity.
- **Higher Initial Load Time:** The initial query might be slower because it retrieves more data at once.
- **More Predictable Performance:** Avoids the cost of additional queries later, which can be beneficial in reducing the overall number of database hits.

